

Selection statements with initializer

Abstract

We propose new versions of the `if` and `switch` statements for C++: `if (init; condition)` and `switch (init; condition)`. These statements simplify common code patterns and help users keep scopes tight.

Contents

1. [Revision history](#)
2. [Before/After](#)
3. [Proposal](#)
4. [Motivation](#)
5. [Alternatives](#)
6. [Discussion](#)
7. [Impact on the Standard](#)
8. [Acknowledgements](#)
9. [Proposed wording](#)
10. [If constexpr](#)

Revision history

- [P0305R0](#): Initial version (post-Jacksonville mailing), “If statement with initializer”.
- P0305R1: This version. Change title to “Selection statements with initializer”. Amend the proposal to also extend `switch` and to support `if constexpr` following EWG guidance.

Before/After

Before the proposal	With the proposal
<pre> { auto p = m.try_emplace(key, value); if (!p.second) { FATAL("Element already registered"); } else { process(p.second); } } </pre>	<pre> if (auto p = m.try_emplace(key, value); !p.second) { FATAL("Element already registered"); } else { process(p.second); } </pre>
<pre> status_code foo() { { status_code c = bar(); if (c != SUCCESS) { return c; } } } // ... </pre>	<pre> status_code foo() { if (status_code c = bar(); c != SUCCESS) { return c; } // ... } </pre>

```
}

```

```
void safe_init() {
    {
        std::lock_guard<std::mutex>
lk(mx_);
        if (v.empty())
            v.push_back(kInitialValue);
    }
}
// ...
}
```

```
{
    Foo gadget(args);
    switch (auto s = gadget.status()) {
        case OK: gadget.zip(); break;
        case Bad: throw
BadFoo(s.message());
    }
}
```

```


```

```
void safe_init() {
    if (std::lock_guard<std::mutex>
lk(mx_); v.empty()) {
        v.push_back(kInitialValue);
    }
    // ...
}
```

(Consider having to move this code around.)

```
switch (Foo gadget(args); auto s =
gadget.status()) {
    case OK: gadget.zip(); break;
    case Bad: throw BadFoo(s.message());
}
```

Proposal

There are three statements in C++, `if`, `for` and `while`, which are all variations on a theme. We propose to make the picture more complete by adding a new form of `if` statement.

Statement	Equivalent to*	Iterations
<code>while(cond) E;</code>	<code>{ while(cond) { E; } }</code>	Repeatedly while <code>cond</code> holds
<code>for (init; cond; inc) E;</code>	<code>{ init; while(cond) { E; inc; } }</code>	Repeatedly while <code>cond</code> holds
<code>if (cond) E;</code>	<code>{ while(cond) { E; break; } }</code>	Once while <code>cond</code> holds
<code>if (cond) E; else F;</code>	(more complex) [†]	Once
<code>if (init; cond) E;</code> new!	<code>{ init; while(cond) { E; break; } }</code>	Once while <code>cond</code> holds
<code>if (init; cond) E; else F;</code> new!	(more complex) [†]	Once

Notes:

*) The “equivalence” ignores the fact that `break` and `continue` have different semantics in loops.
†) The fact that there is no immediate expression of `else` blocks in terms of `while` is due to the absence of a fundamental `while ... else` construction from the language, which would in some sense be a “universal control structure”.

The `switch` statement too uses similar grammar, `switch (cond)`, and we also propose to extend this statement to allow the new form `switch (init; cond)`. This extension was not part of the previous revision of this

proposal, but EWG suggested in Oulu that we may as well offer the same set of syntactic forms for all control structures consistently.

Motivation

The new form of the `if` statement has many uses. Currently, the initializer is either declared before the statement and leaked into the ambient scope, or an explicit scope is used. With the new form, such code can be written more compactly, and the improved scope control makes some erstwhile error-prone constructions a bit more robust:

```
std::map<int, std::string> m;
std::mutex mx;
extern bool shared_flag; // guarded by mx

int demo()
{
    if (auto it = m.find(10); it != m.end()) { return it->size(); }

    if (char buf[10]; std::fgets(buf, 10, stdin)) { m[0] += buf; }

    if (std::lock_guard<std::mutex> lock(mx); shared_flag) { unsafe_ping(); shared_flag = false; }

    if (int s; int count = ReadBytesWithSignal(&s)) { publish(count); raise(s); }

    if (auto keywords = {"if", "for", "while"};
        std::any_of(keywords.begin(), keywords.end(), [&s](const char* kw) { return s == kw;
    })) {
        ERROR("Token must not be a keyword");
    }
}
```

A certain “monadic” style of bubbling up non-success status values also becomes more compact:

```
status_code bar();

status_code foo()
{
    int n = get_value();
    if (status_code c = bar(n); c != status_code::SUCCESS) { return c; }
    if (status_code c = do_more_stuff(); c != status_code::SUCCESS) { return c; }
    return status_code::SUCCESS;
}
```

Alternatives

No extension

There is always the alternative of using an equivalent construction, namely `{ init; if (cond) E; }`. However, this construction is more verbose, and users often use a “lazy approximation” that omits the extra scope. such as:

```
auto it = m.find(10);
if (it != m.end()) { return it->size(); }

// "it" is leaked into the ambient scope.
```

This is often just as good, but in certain cases where the lifetime of the object created in the initializer is important, such as when locking a mutex, forgetting the extra scope may easily have hard-to-diagnose adverse effects. Moreover, the explicit additional scope is brittle and may get lost during refactoring.

A common naming convention is that the length of a name of should correspond to the size of its scope; offering a convenient way to declare names in tight scopes makes it easier to follow such a principle without either introducing unwanted nesting levels or using artificially long names.

Library solution

It is possible to write a library gadget that could contain both initialized values and the result of a boolean expression, but all such attempts have turned up something very unsightly.

Other language extensions

An alternative language extension could be of the form `with (init) if (cond) E;`. Constructions like this exist in other languages. While certainly conceivable, such an extension is more expensive (new keyword, more to teach) and misses out on the opportunity to make an existing facility more consistent.

Discussion

It is often said that C++ is already complex enough, and any additional complexity needs to be carefully justified. We believe that the proposed extension is natural and unsurprising, and thus adds minimal complexity, and perhaps even removes some of the existing differences among the various control flow statements. There is nothing about the local initialization that is specific to loop statements, so having it only on the loop and not on the selection statement seems arbitrary. Had the initializer form of the `if` statement been in the language from the start, it would not have seemed out of place. (At best one might have wondered why `for` is not also spelled `while`, or vice versa.)

For the second point, we would like to consider the advantages of the new form of the `if` statement. Names, lifetimes and scopes are fundamental concepts of C++, and putting the right names into the right scopes is instrumental to understandability and maintainability. The proposed extension is mere syntactic sugar, but it is a convenient tool, readily understood by the reader, that allows the user to keep the scope of auxiliary variables minimal. Current code either requires additional braces to keep scopes minimal, which are visually noisy (taking up valuable indentation levels!) and burdensome to refactor (think of keeping all the lines together), or simply omits the braces, leaking local variables into larger scopes.

Real, existing code bases contain macros that wrap up common idioms (like map lookup and error status propagation), because users find macros the smaller of the two evils compared to leaking lots of local variables or using excessive braces. The proposal removes a common use case for macros.

Note also that the Go language allows initial statements for both `if` and `switch` statements.

Impact on the Standard

This is a core language extension. The newly proposed syntax is ill-formed in the current working draft.

Acknowledgements

Many thanks to Jens Maurer for invaluable help with the wording.

Proposed wording

As part of the wording change, we will rename the grammar production *for-init-statement* to just *init-statement*. In section 3.3.3 [basic.block.scope], change paragraph 4 as follows.

Names declared in the ~~*for-init-statement*~~*init-statement*, the *for-range-declaration*, and the *condition* of `if`, `while`, `for`, and `switch` statements [...]

Append the following grammar to clause 6 [stmt.stmt].

```
declaration-statement
attribute-specifier-seqopt try-block
```

init-statement:

```
expression-statement
simple-declaration
```

In section 6.4 [stmt.select], change the grammar in paragraph 1 as follows.

selection statement:

```
if ( init-statementopt condition ) statement
if ( init-statementopt condition ) statement else statement
switch ( init-statementopt condition ) statement
```

Change the first sentence as follows.

See 8.3 for the optional *attribute-specifier-seq* in a condition. [Note: An *init-statement* ends with a semicolon. – end note] In Clause 6, the term [...]

Insert a new paragraph at the end of subsection 6.4.1 [stmt.if].

An if statement of the form

```
if ( init-statement condition ) statement
```

is equivalent to

```
{
  init-statement
  if ( condition ) statement
}
```

and an if statement of the form

```
if ( init-statement condition ) statement else statement
```

is equivalent to

```
{
  init-statement
  if ( condition ) statement else statement
}
```

except that names declared in the *init-statement* are in the same declarative region as those declared in the *condition*.

Insert a new paragraph at the end of subsection 6.4.2 [stmt.switch].

A switch statement of the form

```
switch ( init-statement condition ) statement
```

is equivalent to

```
{
  init-statement
```

```
switch ( condition ) statement
}
```

except that names declared in the *init-statement* are in the same declarative region as those declared in the *condition*.

In section 6.5 [stmt.iter], modify paragraph 1 as follows.

```
for ( for-init-statementinit-statement conditionopt ; expressionopt ) statement
for ( for-range-declaration : for-range-initializer ) statement
```

for-init-statement:
~~*expression-statement*~~
~~*simple-declaration*~~

for-range-declaration:
*attribute-specifier-seq*_{opt} *decl-specifier-seq* *declarator*

for-range-initializer:
expr-or-braced-init-list

See 8.3 for the optional *attribute-specifier-seq* in a *for-range-declaration*. [Note: ~~A *for-init-statement*~~An *init-statement* ends with a semicolon. – end note]

In section 6.5.3 [stmt.for], change paragraph 1 as follows.

The *for* statement

```
for ( for-init-statementinit-statement conditionopt ; expressionopt ) statement
```

is equivalent to

```
{
  for-init-statementinit-statement
  while ( condition ) {
    statement
    expression ;
  }
}
```

except that names declared in the ~~*for-init-statement*~~*init-statement* are in the same [...]

Change paragraph 3 as follows.

If the ~~*for-init-statement*~~*init-statement* is a declaration [...]

In section 7.1.6.4 [dcl.spec.auto], change paragraph 4 as follows.

[...] in namespace scope (3.3.6), and in ~~a *for-init-statement* (6.5.3)~~*an init-statement* (Clause 6). [...]

If constexpr

In Oulu, EWG also requested that this proposal incorporate the addition of *if constexpr*, which has been approved by EWG and CWG in [P0292r2](#). That proposal and the present one are largely orthogonal, and the facilities of *if constexpr* work just as well with the extended *if* statement from this proposal. For example, the new grammar will be (with changes appertaining to P0292r2 shown as boxed):

selection statement:
if *constexpr*_{opt} (*init-statement*_{opt} *condition*) *statement*
if *constexpr*_{opt} (*init-statement*_{opt} *condition*) *statement* *else statement*

`switch ( init-statementopt condition ) statement`

Drafting note. Care should be taken to apply the new `constexpropt` from P0292r2 to the grammar excerpts in the new paragraph in subsection 6.4.1.

The first sentence of the new wording from P0292r2 needs to be modified to make the reference to the `constexpr` form of the `if` statement clearer:

If the ~~parenthesized condition is prefixed with `constexpr`~~ if statement is of the form `if constexpr`,